

AI-Driven DNA Sequence Classification

Kartavya Mandora
Master in Artificial Intelligence
Long Island University
New York, USA

kartavyappravinsinh.mandora@my.liu.edu

Krupalkumar Patel
Master in Artificial Intelligence
Long Island University
New York, USA

krupalkumaralpeshkumar.patel@my.liu.edu

Swayam Shah
Master in Artificial Intelligence
Long Island University
New York, USA

swayamjigneshbhai.shah@my.liu.edu

Abstract— Accurate identification of splice junctions in DNA sequences is essential for understanding gene expression and genomic structure. This project proposes a hybrid deep learning model combining Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) for multi-class DNA sequence classification. The model classifies sequences into Exon-Intron (EI), Intron-Exon (IE), and Non-splice (N) categories. DNA sequences are represented using both structured encoded features and raw nucleotide sequences, enabling evaluation across different data representations. The CNN component extracts local sequence patterns, while the BiLSTM captures long-range dependencies and contextual relationships. The proposed model is evaluated using standard classification metrics and compared with baseline machine learning models. Experimental results demonstrate that the CNN-BiLSTM approach achieves high classification performance, with consistent accuracy across both encoded and raw-sequence datasets.

Keywords— DNA Sequence Classification, Splice Junction Prediction, Convolutional Neural Network (CNN), Bidirectional Long Short-Term Memory (BiLSTM), Deep Learning, Bioinformatics, Multi-class Classification, One-Hot Encoding, Sequence Modeling

I. INTRODUCTION

DNA sequence classification is a fundamental task in bioinformatics, playing a critical role in understanding gene structure and function. One of the most important problems in this domain is the identification of splice junctions, which define the boundaries between exons and introns in eukaryotic DNA. Accurate detection of these splice sites is essential for proper gene expression analysis, as errors in splicing can lead to serious genetic disorders and diseases. Traditional computational methods for splice site prediction often rely on manually engineered features and classical machine learning models, which may fail to capture complex patterns present in biological sequences.

This approach aims to provide a more accurate and efficient tool for splice junction prediction, supporting genomic research and improving the reliability of computational gene analysis.

II. PRACTICAL APPLICATIONS

A. Applications of DNA Sequence Classification System

➤ Splice Junction Detection

- *Automated Detection*: Detects exon-intron (EI) and intron-exon (IE) boundaries using deep learning models.
- *Improved Accuracy*: Reduces errors compared to traditional machine learning methods.

➤ Gene Structure Analysis

- *Genome Annotation*: Helps identify gene structures and boundaries within DNA sequences.
- *Understanding Gene Expression*: Supports analysis of how genes are formed and processed.

➤ Genomic Data Analysis

- *Large-scale Processing*: Efficiently handles high-throughput sequencing data.
- *Pattern Recognition*: Learns complex biological patterns directly from raw DNA sequences.

➤ Disease Analysis and Genetic Disorder Detection

- *Splicing Error Identification*: Detects abnormal splice junctions that may lead to genetic disorders
- *Research Support*: Helps researchers study the relationship between splicing patterns and diseases such as cancer.

B. Maintaining the Integrity of the Specifications

Maintaining the integrity of the specifications is essential when implementing deep learning models for DNA sequence classification. By using a hybrid CNN + BiLSTM architecture in frameworks such as TensorFlow or PyTorch, the model adheres to a well-defined structure that captures both local sequence patterns and long-range dependencies. Ensuring that data preprocessing steps, including sequence validation, fixed-length window selection, and one-hot encoding, are consistently applied is critical for preserving input quality and biological relevance.

It is also important to carefully tune hyperparameters such as learning rate, number of filters, LSTM units, batch size, and epochs to avoid overfitting and underfitting. Techniques such as dropout, early stopping, and stratified data splitting help maintain model stability and generalization. Additionally, evaluating the model using appropriate multi-class metrics such as accuracy, precision, recall, and F1-score ensures that performance aligns with project objectives. By following these best practices, the system maintains reliability, consistency, and accuracy in splice junction classification.

III. RELATED WORK

DNA sequence classification has been widely studied in the field of bioinformatics, with various traditional and modern approaches proposed over the years. Earlier methods mainly relied on statistical and rule-based techniques, such as sequence alignment tools and pattern matching algorithms. Techniques like BLAST (Basic Local Alignment Search Tool) were commonly used to compare DNA sequences and identify similarities. While these methods are effective, they are often time-consuming and computationally expensive when dealing with large-scale genomic data.

With the advancement of machine learning, researchers started applying classification algorithms such as Support Vector Machines (SVM), Decision Trees, and Naïve Bayes for DNA sequence classification. These models require manual feature extraction techniques like k-mer representation, which converts DNA sequences into numerical form. Although these methods improved performance compared to traditional approaches, they still

depend heavily on feature engineering and may not capture complex patterns in biological data.

More recently, deep learning approaches have gained significant attention due to their ability to automatically extract features from raw data. Models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), including Long Short-Term Memory (LSTM) networks, have been widely used for DNA sequence analysis. CNNs are effective in identifying local patterns within sequences, while RNNs are useful for capturing sequential dependencies. These methods have shown improved accuracy and efficiency compared to traditional machine learning techniques.

Despite these advancements, there are still some challenges in DNA sequence classification, such as handling large datasets, avoiding overfitting, and ensuring model interpretability. In this project, an AI-driven approach is used to classify DNA sequences efficiently by leveraging modern machine learning/deep learning techniques, aiming to achieve better accuracy and performance compared to existing methods.

IV. METHODOLOGY

The methodology of this project focuses on building an AI-driven system for accurate DNA sequence classification. The overall process includes data collection, preprocessing, feature extraction, model training, and evaluation.

A. Dataset Description

The dataset (2 datasets) used in this project consists of DNA sequences, where each sequence is composed of nucleotide bases such as Adenine (A), Thymine (T), Cytosine (C), and Guanine (G). Each sequence is associated with a specific class label, which represents its biological category. The dataset is divided into training and testing sets to evaluate the performance of the model.

The dataset contains labelled DNA sequences categorized into EI, IE, and N classes, with each sequence having a fixed length used for model training.

Table 1: Labels For Both Datasets

Label	Old Dataset	New Dataset
0	N (Neither / Non-splice)	EI (Exon-Intron)
1	EI (Exon-Intron)	IE (Intron-Exon)
2	IE (Intron-Exon)	N (Non-splice)

The label representation differs between the original and extended datasets despite representing the same biological classification problem. In the StatLog dataset, label 0 corresponds to non-splice regions, label 1 represents exon-intron boundaries, and label 2 corresponds to intron-exon boundaries. In contrast, the extended dataset follows a different ordering where label 0 represents exon-intron, label 1 represents intron-exon, and label 2 corresponds to non-splice regions. This difference was explicitly handled during preprocessing and evaluation to ensure consistent interpretation of class labels across both datasets.

B. Dataset Evaluation

To further evaluate the robustness and scalability of the proposed CNN-BiLSTM model, an additional experiment was conducted using a larger DNA splice site dataset obtained from the Hugging Face repository. This dataset

consists of 45,619 DNA sequences, each of fixed length 400, with corresponding multi-class labels. Unlike the initial dataset, which was already numerically encoded, this dataset contains raw nucleotide sequences that required preprocessing through sequence normalization and one-hot encoding. Each nucleotide (A, C, G, T) was represented as a 4-dimensional binary vector, resulting in an input representation of shape (400, 4). This extended experiment allows the model to be evaluated on more realistic and large-scale genomic data, providing deeper insight into its generalization capability.

Table 2: Comparison of Datasets

Feature	Old Dataset	New Dataset
Samples	3,190	45,619
Format	Encoded (180 features)	Raw DNA sequence
Input Shape	(60, 3)	(400, 4)
Preprocessing	Minimal	One-hot encoding
Complexity	Low	High
Realism	Moderate	High

The comparison highlights that the new dataset introduces higher complexity and realism, requiring additional preprocessing steps and enabling the model to learn more generalized sequence representations. This transition from structured numerical features to raw biological sequences significantly enhances the applicability of the proposed model in real-world genomic analysis.

C. Data Preprocessing

The preprocessing stage differs significantly between the two datasets used in this study due to their underlying representations. For the original StatLog dataset, the DNA sequences are already provided in a pre-encoded numerical format consisting of 180 binary features. These features correspond to 60 nucleotide positions, each represented using three binary indicators. Therefore, minimal preprocessing is required, and the input is directly reshaped into a structured format of (60, 3) to preserve positional relationships before being fed into the CNN-BiLSTM model.

In contrast, the extended dataset obtained from the Hugging Face repository consists of raw DNA sequences represented as strings of nucleotides (A, C, G, T). Since deep learning models cannot process string data directly, a dedicated preprocessing pipeline was implemented. First, all sequences were converted to uppercase and normalized to a fixed length of 400 nucleotides. This was achieved by padding shorter sequences with a neutral character ('N') and truncating longer sequences to maintain uniform input dimensions. The use of a fixed-length representation ensures compatibility with convolutional and recurrent layers while preserving sequence structure.

Following normalization, each nucleotide was transformed into a one-hot encoded vector of length four, where A, C, G, and T are represented as distinct binary patterns. This encoding converts each DNA sequence into a three-dimensional tensor of shape (400, 4), enabling the CNN layers to extract local patterns and the BiLSTM layers to capture long-range dependencies. This transformation is

Note: All the figures in comparison part on the left is the Older Dataset ("dna.csv") and on the right Newer Dataset (raw sequences)

critical as it preserves both positional and categorical information of nucleotides without introducing numerical bias.

To ensure reproducibility and consistency across experiments, a fixed random seed was applied to all preprocessing and data splitting operations. The dataset was then divided into training (80%), validation (10%), and testing (10%) subsets using stratified sampling to maintain class distribution across splits. Additionally, class weights were computed to address class imbalance, particularly due to the higher proportion of non-splice sequences in the extended dataset.

D. Feature Extraction / Encoding

To transform DNA sequences into numerical representations, encoding methods such as k-mer representation or one-hot encoding are used. In this approach, each nucleotide (A, T, C, G) is converted into a numerical format that can be understood by the model. This step is important because it helps the model learn patterns and relationships within the DNA sequences.

E. Model Selection

In this project, a hybrid CNN-BiLSTM model is used for DNA sequence classification. The CNN component extracts local features from DNA sequences, while the BiLSTM captures sequential dependencies, resulting in improved classification performance.

F. Training Process

The dataset is split into training and testing sets. The model is trained using the training data, where it learns the relationship between input sequences and their corresponding labels. During training, optimization techniques are applied to minimize the loss function and improve accuracy. Hyperparameters such as learning rate, number of epochs, and batch size are adjusted to achieve better performance.

G. Model Evaluation

After training, the model is evaluated using the test dataset. Performance metrics such as accuracy are used to measure how well the model classifies unseen DNA sequences. The evaluation helps in understanding the effectiveness and reliability of the proposed system.

V. SYSTEM ARCHITECTURE / MODEL DESIGN

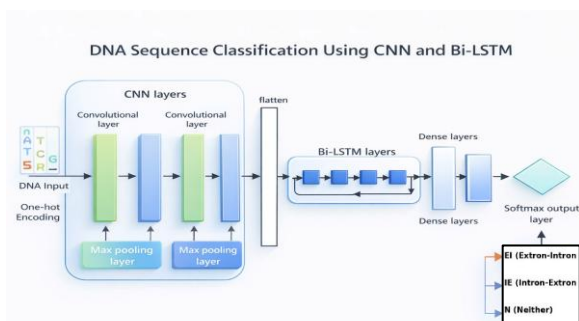


Figure 1: Model Architecture

A. Convolutional Neural Network (CNN) Architecture

The Convolutional Neural Network (CNN) is used as the first component of the proposed model to extract local features from DNA sequences. Initially, the input DNA sequences are converted into numerical form using one-hot encoding, where each nucleotide (A, T, C, G) is represented as a binary vector. This encoded input is then fed into the CNN layers.

The CNN architecture consists of multiple convolutional layers followed by max-pooling layers. The convolutional layers apply filters to the input data to capture important local patterns and motifs within the DNA sequences. These filters help in identifying meaningful biological features such as recurring nucleotide patterns.

After each convolutional operation, a max-pooling layer is applied to reduce the dimensionality of the feature maps while retaining the most important information. This helps in reducing computational complexity and prevents overfitting.

The output of the CNN layers is then flattened into a one-dimensional vector, which serves as input to the next stage of the model. Overall, the CNN component plays a crucial role in extracting spatial features from DNA sequences effectively.

B. Bidirectional Long Short-Term Memory (Bi-LSTM)

The Bidirectional Long Short-Term Memory (Bi-LSTM) network is used after the CNN layers to capture sequential dependencies in the DNA data. Unlike traditional neural networks, LSTM is specifically designed to handle sequential data and retain information over long sequences.

In this model, a bidirectional LSTM is used, which processes the sequence in both forward and backward directions. This allows the model to capture context from both past and future positions in the DNA sequence, leading to a better understanding of the relationships between nucleotides.

The flattened output from the CNN layers is provided as input to the Bi-LSTM layers. These layers learn long-term dependencies and sequential patterns that are not captured by CNN alone. This combination of CNN and Bi-LSTM improves the overall feature representation of the data.

After processing through the Bi-LSTM layers, the output is passed to fully connected (dense) layers, which perform the final classification. A softmax activation function is used in the output layer to classify the DNA sequences into different categories such as EI, IE, and N splice junction categories.

VI. IMPLEMENTATION

A. Software and Tools Used

The proposed model is implemented using Python with TensorFlow and Keras for deep learning. Additional libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn are used for data preprocessing and evaluation. The model is executed on T4 GPU.

The implementation follows a structured and reproducible coding pipeline. The dataset is first loaded and validated to ensure consistency in sequence format and labels. For the

Note: All the figures in comparison part on the left is the Older Dataset ("dna.csv") and on the right Newer Dataset (raw sequences)

extended dataset, raw DNA sequences are normalized to a fixed length and converted into one-hot encoded representations to preserve nucleotide-level information. Labels are encoded using Label Encoder and transformed into categorical format for multi-class classification. The dataset is then split into training, validation, and testing sets using stratified sampling to maintain class balance. The CNN-BiLSTM model is trained using TensorFlow with early stopping, learning rate scheduling, and model checkpointing to ensure stable convergence. Finally, performance is evaluated using multiple metrics including accuracy, precision, recall, F1-score, ROC curves, and Precision-Recall curves.

B. Data Preparation and Splitting

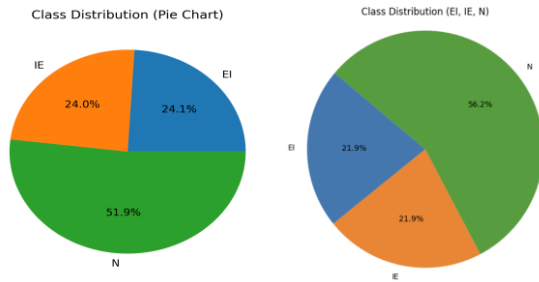


Figure 2: Dataset Class Distribution (Old vs New)

The preprocessing and data preparation steps differ between the two datasets. For the original encoded dataset, the input features are reshaped into a structured format of (60, 3), representing nucleotide positions. In contrast, for the extended raw-sequence dataset, DNA sequences are first normalized to a fixed length of 400 and then converted into one-hot encoded representations, resulting in an input shape of (400, 4). This ensures that both datasets are transformed into formats compatible with the CNN-BiLSTM architecture while preserving biological sequence information.

C. Model Training Setup

The CNN-BiLSTM model is trained using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy loss. The training is performed for a maximum of **50 epochs** with a **batch size of 32**. Early Stopping (patience = 8), Reduce LR On Plateau, and Model Checkpoint are used to improve training performance and prevent overfitting.

The CNN-BiLSTM model was optimized using carefully selected hyperparameters to ensure efficient learning and good generalization. The model was trained using the Adam optimizer with a learning rate of 0.001 and a batch size of 32 for up to 50 epochs, with early stopping applied to prevent overfitting. The convolutional layers used 64 and 128 filters with kernel sizes of 3 and 5 to effectively capture both local nucleotide patterns and long-range sequence dependencies, while the BiLSTM layer with 64 units modeled long-range dependencies. A dropout rate of 0.3 was applied to improve generalization. The architecture includes two dense layers: a hidden layer with 64 neurons for feature learning and an output layer with softmax activation for multi-class classification. The model was trained using categorical cross-entropy loss, appropriate for splice junction prediction tasks.

The selection of hyperparameters was guided by experimental validation and performance analysis. A comparison of different BiLSTM unit sizes (32, 64, and 128) showed that although 128 units achieved the highest validation accuracy, 64 units were selected as the final configuration to balance performance, computational efficiency, and risk of overfitting. Smaller configurations underfit the data, while larger configurations increased computational cost without significant improvement. Similarly, the use of 64 and 128 convolutional filters enables the model to capture both simple and complex sequence patterns. The chosen learning rate, batch size, and dropout values ensure stable training and effective regularization. To ensure reproducibility, a fixed random seed (42) was applied across NumPy, TensorFlow, and data splitting operations.

D. Evaluation and Baseline Comparison

The model is evaluated on the test dataset using accuracy, precision, recall, and F1-score. A confusion matrix and classification report are generated for detailed analysis. Additionally, baseline models such as SVM, and Random Forest (200 estimators) are implemented for performance comparison.

VII. RESULTS AND DISCUSSION

The performance of the proposed CNN-BiLSTM model was evaluated on the test dataset using multiple evaluation metrics, including accuracy, precision, recall, and F1-score. The model was trained for a maximum of 50 epochs with a batch size of 32, and early stopping was applied to prevent overfitting.

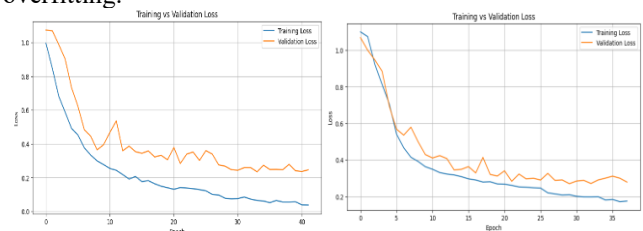


Figure 3: Training vs Validation Loss (Old & New)

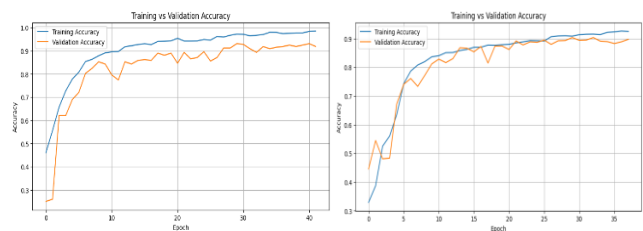


Figure 4: Training vs Validation Accuracy (Old vs New)

The results show that the model is able to effectively classify DNA sequences into their respective classes. The convolutional layers successfully extract local patterns from the sequences, while the bidirectional LSTM captures long-term dependencies, resulting in improved classification performance.

The proposed CNN-BiLSTM model demonstrates strong performance in DNA sequence classification. This indicates that the proposed architecture effectively balances feature extraction and sequence modeling, which is critical for splice site detection tasks.

Note: All the figures in comparison part on the left is the Older Dataset ("dna.csv") and on the right Newer Dataset (raw sequences)

The performance of the proposed CNN-BiLSTM model was evaluated using standard classification metrics. The model achieved a **training accuracy of 98.43%(Old), 92.51%(New)** and a **testing accuracy of 94.04%(Old), 90.50%(New)**, indicating good generalization performance. Additionally, the model obtained a **precision of 93.48%(Old), 88.40%(New)**, **recall of 94.32%(Old), 92.70%(New)** and an **F1-score of 93.86%(Old), 90.20%(New)**, demonstrating balanced and reliable classification across all classes.

Table 3: Comparison with Baseline Models

Metric	Old Dataset	New Dataset
Accuracy	94.04%	90.50%
Precision (Macro)	93.48%	88.40%
Recall (Macro)	94.32%	92.70%
F1 Score (Macro)	93.86%	90.20%

The comparison between datasets reveals that while the model achieves higher accuracy on the structured encoded dataset, performance slightly decreases on the raw-sequence dataset due to increased complexity and variability. However, the results remain strong, demonstrating the model’s ability to generalize effectively to real-world DNA sequence data.

Table 4: Comparison between All Models

Model	Accuracy (Old)	Accuracy (New)
SVM	95.92%	79.15%
Random Forest	96.24%	73.69%
CNN + BiLSTM	94.04%	90.50%

The comparison clearly demonstrates that while traditional machine learning models perform competitively on structured encoded data, the CNN-BiLSTM model significantly outperforms them on raw DNA sequences due to its ability to learn complex sequence dependencies.

The slight performance drop observed on the raw-sequence dataset highlights the increased complexity of learning directly from biological sequences compared to structured encoded features. The ROC curves demonstrate excellent discriminative capability, with AUC values of 1.00, 0.99, and 0.98 (Old) & 0.99, 0.99, and 0.98 (New) for the three classes, confirming the model’s ability to distinguish between splice junction categories with high confidence. Similarly, the Precision-Recall curves show high average precision values, particularly for the EI and N classes, highlighting the model’s robustness in handling class imbalance and correctly identifying splice boundaries. The classification report further indicates consistent performance across all classes, with particularly high precision for the non-splice class and strong recall for splice junction classes. Overall, these results validate that the hybrid CNN-BiLSTM architecture effectively captures both local nucleotide patterns and long-range dependencies, leading to improved generalization and reliable splice junction prediction. The confusion matrix indicates that the CNN-BiLSTM model achieves high classification accuracy, with most predictions correctly aligned along the diagonal, demonstrating strong performance across all classes. Minor misclassifications are observed primarily between the EI and IE classes, which is

expected due to their biological similarity, while the non-splice class (N) is predicted with the highest precision.

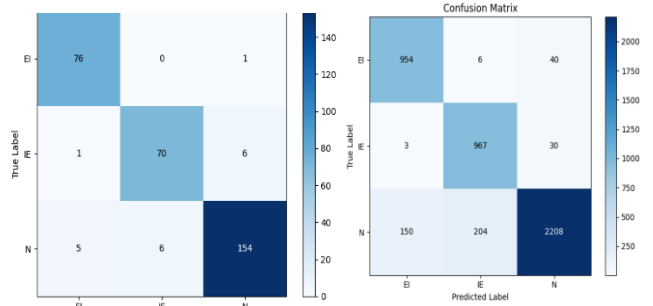


Figure 5: Confusion Matrix of CNN-BiLSTM Model (Old Vs New)

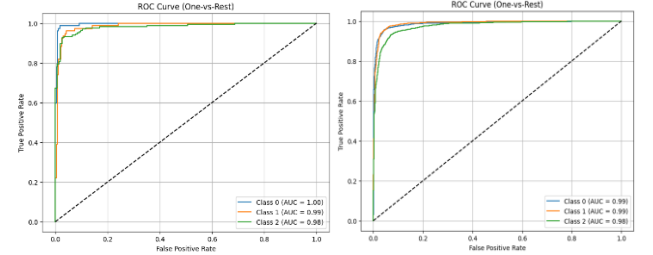


Figure 6: ROC curve showing model discrimination performance (Old vs New)

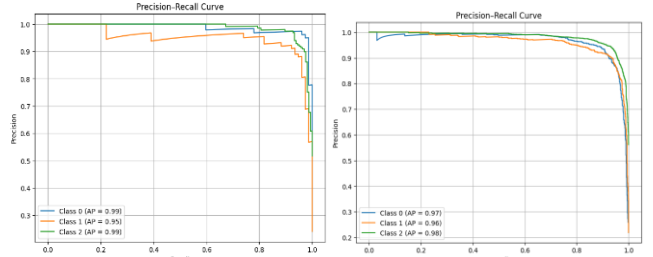


Figure 7: Precision-Recall curve (Old vs New)

Additionally, techniques such as dropout, class weighting, and learning rate adjustment were applied to improve generalization and handle class imbalance. These strategies helped in reducing overfitting and improving the robustness of the model.

To improve interpretability, the learned weights of the first convolutional layer were visualized. The filters exhibit diverse patterns across kernel positions, indicating that the CNN component learns multiple local sequence motifs. These learned representations suggest that the model is effectively capturing biologically meaningful nucleotide patterns relevant to splice site identification. This aligns with findings from the SpliceRover study, where convolutional filters are shown to correspond to interpretable genomic motifs such as donor and acceptor signals.

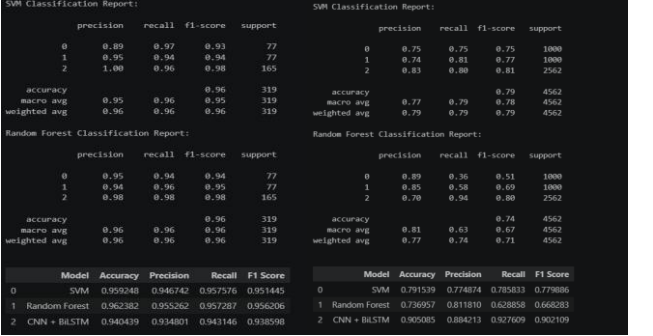


Figure 8: SVM & Random Forest Report (Old vs New)

Note: All the figures in comparison part on the left is the Older Dataset (“dna.csv”) and on the right Newer Dataset (raw sequences)

The performance of the baseline models, Support Vector Machine (SVM) and Random Forest, was evaluated on both the encoded dataset and the extended raw-sequence dataset to assess their effectiveness in DNA sequence classification. For the extended dataset, SVM achieved an accuracy of approximately 79%, demonstrating moderate performance with relatively balanced precision and recall across classes. In contrast, Random Forest achieved a lower accuracy of around 74%, with notably poor recall for certain classes, indicating difficulty in capturing complex sequence patterns. On the original encoded dataset, both models performed significantly better, achieving accuracies close to 96%, which reflects the simplicity and structured nature of the pre-encoded features. However, despite this improvement, both classical models were consistently outperformed by the CNN-BiLSTM model, particularly on the extended dataset where deep learning demonstrated superior ability to learn complex nucleotide dependencies. These results highlight that while traditional machine learning models perform well on structured datasets, they struggle with raw biological sequences, whereas deep learning approaches provide better generalization and robustness.

Total misclassified samples: 19

Index	True_Label	Predicted_Label	Index	True_Label	Predicted_Label		
0	7	IE	N	0	6	N	EI
1	30	EI	N	1	20	N	EI
2	39	N	IE	2	31	N	IE
3	63	N	EI	3	34	N	IE
4	122	N	EI	4	48	N	EI
5	138	IE	N	5	55	N	IE
6	143	IE	N	6	59	IE	N
7	149	N	EI	7	94	N	IE
8	165	N	IE	8	97	N	EI
9	181	IE	EI	9	98	N	EI
10	206	N	IE	10	111	N	IE
11	211	IE	N	11	114	N	EI
12	234	IE	N	12	122	IE	N
13	235	N	EI	13	124	EI	N
14	257	N	IE	14	141	N	IE
15	258	N	EI	15	144	N	IE
16	269	N	IE	16	159	N	IE
17	272	IE	N	17	164	N	EI
18	289	N	IE	18	178	N	IE
			19	187	N	IE	

Total misclassified samples: 433

Misclassification summary:

True_Label	Predicted_Label	Count	
0	EI	N	1
1	IE	EI	1
2	IE	N	6
3	N	EI	5
4	N	IE	6

Misclassification summary:

True_Label	Predicted_Label	Count	
0	EI	IE	6
1	EI	N	40
2	IE	EI	3
3	IE	N	30
4	N	EI	150
5	N	IE	204

Figure 9: Misclassified Samples (Old Vs New)

A detailed misclassification analysis was conducted to better understand the prediction behavior of the model across both datasets. For the original encoded dataset, the number of misclassified samples is relatively low, indicating strong model performance and clear separation between classes. The majority of errors occur between the EI and IE classes, which is expected due to their biological similarity as splice boundary regions. In contrast, the extended raw-sequence dataset shows a higher number of misclassified samples, with most errors originating from the non-splice (N) class being incorrectly predicted as EI or IE. This suggests that the model faces greater difficulty distinguishing non-splice regions from true splice sites when working with raw sequence data. Additionally, occasional confusion between EI and IE is observed, reflecting overlapping sequence patterns near

Note: All the figures in comparison part on the left is the Older Dataset ("dna.csv") and on the right Newer Dataset (raw sequences)

splice junctions. Overall, this analysis highlights that while the model performs robustly on structured encoded data, classification becomes more challenging with raw DNA sequences due to increased variability and complexity, reinforcing the importance of deep learning architectures for capturing nuanced biological patterns.

BiLSTM Units			Best Validation Accuracy		
0	32	0.880878	0	32	0.794608
1	64	0.830721	1	64	0.855984
2	128	0.890282	2	128	0.865629

Figure 10: Hyperparameters Tuning Results (Old vs New)

A hyperparameter analysis was conducted to evaluate the impact of different BiLSTM unit sizes on model performance across both datasets. For the original encoded dataset, the model achieved its best validation accuracy with 128 units, followed closely by 32 units, while 64 units resulted in comparatively lower performance. Similarly, for the extended raw-sequence dataset, the highest validation accuracy was also observed with 128 units, indicating that larger BiLSTM configurations are more effective in capturing complex sequence dependencies. However, increasing the number of units also introduces higher computational cost and potential risk of overfitting. Therefore, a moderate configuration of 64 units was selected for the final model as it provides a balance between performance, generalization, and computational efficiency. This experiment demonstrates that while larger architectures can improve accuracy, an optimal trade-off is necessary for stable and efficient model training.

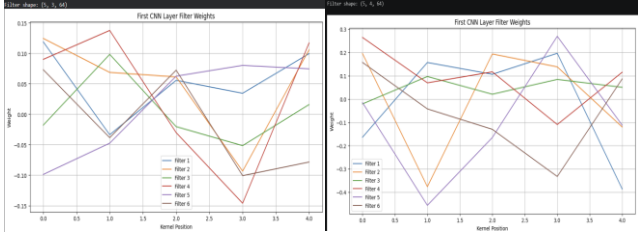


Figure 11: CNN Filter Interpretation (Old vs New)

To enhance model interpretability, the learned weights of the first convolutional layer were analyzed for both datasets. The visualization shows that different filters exhibit distinct weight patterns across kernel positions, indicating that the CNN component learns diverse local sequence features. For the encoded dataset, the filter weights appear more structured and stable, reflecting the consistency of the pre-encoded input features. In contrast, for the raw-sequence dataset, the filters display greater variability and fluctuation in weight values, suggesting that the model is adapting to more complex and less structured input representations. The presence of both positive and negative weights across kernel positions indicates that the model selectively emphasizes and suppresses specific nucleotide patterns. Overall, these observations confirm that the CNN layer effectively acts as a motif detector, learning meaningful local patterns that contribute to splice site classification, consistent with findings in prior studies such as SpliceRover. This demonstrates that the model is not only accurate but also interpretable, an important requirement in bioinformatics applications.

Despite the strong performance achieved, the model has certain limitations. The performance on the raw-sequence dataset is slightly lower compared to the encoded dataset, indicating that learning directly from biological sequences introduces additional complexity. Furthermore, the model occasionally struggles to distinguish non-splice regions from splice boundaries, as observed in the misclassification analysis. These limitations highlight the challenges associated with real-world genomic data and suggest that further improvements in feature representation and model architecture could enhance performance.

VIII. CONCLUSION & FUTURE WORK

The proposed CNN-BiLSTM model demonstrates strong performance in splice junction classification, achieving a training accuracy of 98.43% on the encoded dataset and 92.51% on the raw-sequence dataset, with corresponding testing accuracy of 94.04% and 90.50%. These results indicate effective generalization with minimal overfitting across datasets of varying complexity. The integration of convolutional layers and bidirectional LSTM enables the model to capture both local nucleotide patterns and long-range sequence dependencies, resulting in balanced and reliable classification across EI, IE, and N classes. Comprehensive evaluation using accuracy, precision, recall, F1-score, ROC curves, and Precision-Recall curves confirms the robustness and high discriminative capability of the model. Furthermore, the comparative analysis between structured encoded and raw-sequence datasets highlights the model's adaptability to different data representations, reinforcing its applicability in real-world genomic analysis. These findings highlight the effectiveness of hybrid deep learning architectures in handling both structured and unstructured genomic data for splice junction prediction.

IX. ACKNOWLEDGMENT

We would like to express our sincere gratitude to the professor, **Dr. Reda Nacif Elalaoui**, for his valuable guidance, continuous support, and insightful feedback throughout the course of this project. We also acknowledge the contributions and collaboration of all team members whose efforts made this work possible.

Additionally, we would like to thank the providers of the datasets and research resources that supported this study. Finally, we express our appreciation to our families and friends for their constant encouragement and support.

X. REFERENCES

- [1] Alipanahi, B., Delong, A., Weirauch, M. T., & Frey, B. J. (2015). Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning. *Nature biotechnology*, 33(8), 831-838.
- [2] Bengio, Y. (2009). Learning deep architectures for AI. *Foundations and Trends® in Finance*, 2(1), 1-127.
- [3] Bari, A. T. M. G., Reaz, M. R., & Jeong, B. S. (2014). Effective DNA encoding for splice site prediction using SVM. *MATCH Commun Math Comput Chem*, 71, 241-258.
- [4] Degroove, S., Saeys, Y., De Baets, B., Rouzé, P., & Van de Peer, Y. (2005). SpliceMachine: predicting splice sites from high-dimensional local context representations. *Bioinformatics*, 21(8), 1332-1338.
- [5] Dakhli, A., & Bellil, W. (2016). Wavelet neural networks for DNA sequence classification using the genetic algorithms and the least trimmed square. *Procedia Computer Science*, 96, 418-427.
- [6] Jaganathan, K., Panagiotopoulou, S. K., McRae, J. F., Darbandi, S. F., Knowles, D., Li, Y. I., ... & Farh, K. K. H. (2019). Predicting splicing from primary sequence with deep learning. *Cell*, 176(3), 535-548.
- [7] Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*.
- [8] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.
- [9] Lee, T., & Yoon, S. (2015, June). Boosted categorical restricted Boltzmann machine for computational prediction of splice junctions. In *International conference on machine learning* (pp. 2483-2492). PMLR.
- [10] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, 521(7553), 436-444.
- [11] Rizzo, R., Fiannaca, A., La Rosa, M., & Urso, A. (2015, September). A deep learning approach to DNA sequence classification. In *International meeting on computational intelligence methods for bioinformatics and biostatistics* (pp. 129-140). Cham: Springer International Publishing.
- [12] Shrikumar, A., Greenside, P., & Kundaje, A. (2017, July). Learning important features through propagating activation differences. In *International conference on machine learning* (pp. 3145-3153). PMIR.
- [13] Sonnenburg, S., Schweikert, G., Philips, P., Behr, J., & Rätsch, G. (2007). Accurate splice site prediction using support vector machines. *BMC bioinformatics*, 8(Suppl 10), S7.
- [14] Zeng, H., Edwards, M. D., Liu, G., & Gifford, D. K. (2016). Convolutional neural network architectures for predicting DNA-protein binding. *Bioinformatics*, 32(12), i121-i127.
- [15] Zuallaert, J., Godin, F., Kim, M., Soete, A., Saeys, Y., & De Neve, W. (2018). SpliceRover: interpretable convolutional neural networks for improved splice site prediction. *Bioinformatics*, 34(24), 4180-4188.

Note: All the figures in comparison part on the left is the Older Dataset ("dna.csv") and on the right Newer Dataset (raw sequences)